ARM BHARAT AI-SoC CHALLENGE 2026–27

Problem Statement 3: Cybersecurity and Digital Safety

KAVACH

An On-Device, Multilingual Scam-Call Shield for Indian Phone Users with Llama 3.2 1B on Arm CPUs

A Phase 1 Technical Report

submitted to the Arm Bharat AI-SoC Challenge 2026–27

Submitted by

Siddhant Singh

Bogavalli Sai Akhilash

Shivansh Bose

Under the guidance of

Dr. Rahul Ratnakumar

Faculty Mentor

NOVEMBER 2026

ABSTRACT

Phone fraud in India has moved from single messages to live, orchestrated conversations. In a typical “digital arrest” call the fraudster claims to be from a police cyber-crime branch, says that a parcel booked on the victim’s Aadhaar contains drugs, forbids the victim to tell anyone, and finally asks for a transfer to a so-called RBI safe account. Every step is spoken, and the payment is made while the call is still running. Protection that reads only SMS or links arrives after the money has gone, and protection that uploads call audio to a server creates a new privacy risk of its own.

This report presents KAVACH (Sanskrit for armour), a single Android application that listens to a call the user has put on speaker, recognises Hindi, Hinglish and English speech on the phone itself, and warns the user during the call, with no internet permission at all. Audio is cut into sentences by a voice-activity detector and recognised by two int8 CTC speech models: AI4Bharat IndicConformer for Hindi and NVIDIA Parakeet 110M for English. An Auto mode runs both models on every sentence and keeps the transcript whose words are real words of its language, so a caller may switch languages mid-call. A detector that takes about 4 ms per sentence, made of a cue lexicon, a hashed linear classifier and a multi-channel kill chain, raises CAUTION or STOP. A Llama 3.2 1B model, fine-tuned with LoRA on KAVACH’s scam playbook, quantised to 4 bits and run with ExecuTorch on the Arm CPU, then explains the alert in the caller’s language. Two further on-device pipelines, Shield (messages, links, apps, screen sharing and payments) and the optional Network Watch app (app traffic, DNS look-ups and risky devices on the home Wi-Fi), feed the same kill chain.

On a mid-range Xiaomi phone with a Snapdragon 7s Gen 2 processor, both speech models together run about five times faster than real time, a one-minute listening run drew 385 mA for the whole phone (about 7.6 % of the battery per hour if it were sustained) with no rise in temperature, and Llama decodes 11.5 to 11.9 tokens per second. In three real cellular calls from an unknown number, a Hindi digital-arrest scam and an English fake-KYC scam reached STOP during the call and a genuine Hindi bank fraud-check call raised no warning. On host evaluation the detector reached STOP on 93.4 % of scams in 1,000 unseen synthetic calls, and no genuine call reached STOP in any evaluated set (0 of 489 synthetic and 0 of 24 human-written genuine calls; too few human-written calls to prove a rate). A cross-channel ablation shows that joining channels adds STOPs at the risky action, not earlier warnings, and can blame a genuine call for an unrelated scam message. Replacing the small streaming Hindi model with IndicConformer raised telephone-band Hindi detection from 0 % to 58 % STOP. LoRA fine-tuning raised Llama’s valid-verdict rate from 71 % to 100 % and its scam accuracy from 38 % to 89 %.

Tools used: Kotlin and the Android SDK, Python 3, PyTorch, Hugging Face Transformers and PEFT, ExecuTorch 1.4 with XNNPACK and Arm KleidiAI, sherpa-onnx, Silero VAD, Piper TTS, scikit-learn, GitHub Actions, Google Colab (T4 GPU).

CONTENTS

ABSTRACT ..... **ii**
LIST OF TABLES ..... **vi**
LIST OF FIGURES ..... **vii**
LIST OF ABBREVIATIONS ..... **viii**
CHAPTER 1 INTRODUCTION **1**
1.1 Introduction 1
1.2 Background: Phone Scams in India 1
1.3 Anatomy of a Phone Scam 1
1.4 Why the Protection Must Run on the Phone 2
1.5 Problem Statement 2
1.6 Objectives 3
1.7 Target Specifications 3
1.8 Organisation of the Report 4
CHAPTER 2 BACKGROUND AND RELATED WORK **5**
2.1 Introduction 5
2.2 On-Device Speech Recognition 5
2.3 Small Language Models on Phones 5
2.4 Parameter-Efficient Fine-Tuning with LoRA 6
2.5 Lightweight Text Classification 6
2.6 Android Rules for Call Audio 6
2.7 Existing Approaches and the Gap 6
2.8 Summary 7
CHAPTER 3 METHODOLOGY AND SYSTEM DESIGN **8**
3.1 Introduction 8
3.2 Overall Architecture 8
3.3 Listening to a Call on the Same Phone 10
3.4 Speech Front-End 12
3.5 The Always-On Detector 13
3.6 The Llama Advisor 16
3.7 Shield 18
3.8 Network Watch 18
3.9 User Interface 19
3.10 Privacy and Security Design 19
3.11 Tools and Technologies 20
3.12 Summary 20
CHAPTER 4 IMPLEMENTATION **21**
4.1 Introduction 21
4.2 Software Structure 21

4.3 Android Application..... 21
4.4 Keeping Python and Kotlin Identical..... 22
4.5 Continuous Integration..... 22
4.6 Training Data..... 22
4.7 Training and Exporting Llama..... 23
4.8 Deployment to the Phone..... 23
4.9 Summary..... 23
CHAPTER 5 RESULTS AND DISCUSSION..... 24
5.1 Introduction..... 24
5.2 Evaluation Sets and Metrics..... 24
5.3 Text Classifiers..... 24
5.4 Call-Level Detection..... 25
5.5 Does Cross-Channel Fusion Help?..... 26
5.6 Speech in Noisy Rooms..... 27
5.7 Hindi Speech Recognition..... 27
5.8 Auto Language Choice..... 29
5.9 Tests on the Phone with Replayed Audio..... 29
5.10 Real Phone Calls..... 29
5.11 The Llama Advisor..... 31
5.12 On-Phone Benchmark..... 33
5.13 Network Watch..... 34
5.14 Problems Found and Fixed..... 35
5.15 Targets and Achievements..... 36
5.16 Summary..... 36
CHAPTER 6 LIMITATIONS, ETHICS AND THREAT MODEL..... 37
6.1 Introduction..... 37
6.2 Limitations..... 37
6.3 Threat Model..... 38
6.4 Ethics and Privacy..... 38
6.5 Summary..... 38
CHAPTER 7 CONCLUSION AND FUTURE WORK..... 39
7.1 Conclusion..... 39
7.2 Future Work (Phase 2)..... 39
REFERENCES..... 41
ANNEXURE A: INSTALLING AND SETTING UP KAVACH..... 43
ANNEXURE B: REPRODUCING THE RESULTS..... 44
ANNEXURE C: LLAMA OUTPUT CONTRACT..... 45
ANNEXURE D: BENCHMARK OUTPUT FIELDS..... 46
ANNEXURE E: TEAM CONTRIBUTIONS..... 47

LIST OF TABLES

Table 1.1: Scam types in the KAVACH playbook....2
Table 1.2: Target specifications of KAVACH....3
Table 2.1: Categories of scam protection compared with KAVACH....7
Table 3.1: Stages of the speech front-end....12
Table 3.2: Cue categories of the lexicon....13
Table 3.3: Rules of the kill chain....14
Table 3.4: Fields of Llama's JSON verdict....16
Table 3.5: LoRA fine-tuning configuration....17
Table 3.6: Privacy measures and how they are enforced....19
Table 3.7: Tools and technologies used....20
Table 4.1: Main parts of the repository....21
Table 4.2: Data used for training and evaluation....22
Table 5.1: Metrics used in this chapter....24
Table 5.2: Text classifiers on held-out test splits....24
Table 5.3: Call-level results (rules only vs shipped detector)....25
Table 5.4: Fusion ablation, scam sessions: STOPs out of 40 per template....26
Table 5.5: Fusion ablation, genuine sessions: STOPs out of 40 (in brackets: STOPs that blame the call)....26
Table 5.6: English recogniser choices in noisy rooms (host)....27
Table 5.7: Streaming Hindi model before and after lexicon and classifier fixes....28
Table 5.8: Streaming Hindi model vs IndicConformer on the same 100 calls....28
Table 5.9: Per-sentence language choice in Auto mode....29
Table 5.10: Replayed test files on the phone (12 November 2026)....29
Table 5.11: Real cellular calls on one phone (12 November 2026)....30
Table 5.12: Checks on the Llama export pipeline....31
Table 5.13: Llama 3.2 1B before and after KAVACH LoRA fine-tuning....31
Table 5.14: On-phone benchmark (12 November 2026)....33
Table 5.15: Network Watch results....34
Table 5.16: Problems found during testing and how they were fixed....35
Table 5.17: Target specifications compared with what was achieved....36
Table 6.1: Ways a scammer may try to avoid KAVACH and the defences....38
Table 7.1: Phase 2 plan....39
Table D.1: Main fields of the benchmark JSON file....46
Table E.1: Main areas of work....47

LIST OF FIGURES

Figure 3.1: Overall architecture of KAVACH: three on-device pipelines feed one kill chain; Llama explains alerts.....9
Figure 3.2: Flow of one protected call from an unknown number.....11
Figure 3.3: States of the kill chain from first contact to STOP.....15
Figure 3.4: Sequence from a spoken sentence to the warning and Llama's explanation.....16
Figure 3.5: Main screens of the KAVACH app.....19
Figure 5.1: Hindi scam calls reaching STOP: streaming model vs IndicConformer.....28
Figure 5.2: Screens during the real calls: Hindi digital arrest, English fake KYC and the genuine bank call.....30
Figure 5.3: Llama quality metrics before and after LoRA fine-tuning.....32
Figure 5.4: Llama 3.2 1B explaining a Hindi digital-arrest call on the phone.....33
Figure 5.5: Benchmark screen on the phone.....34

LIST OF ABBREVIATIONS

Abbreviation	Meaning
ASR	Automatic Speech Recognition
CTC	Connectionist Temporal Classification
VAD	Voice Activity Detection
LLM	Large Language Model
LoRA	Low-Rank Adaptation
RTF	Real-Time Factor (processing time / audio time)
TTFT	Time To First Token
KV cache	Key-Value cache of a transformer
int8 / 4-bit	8-bit / 4-bit integer quantisation
8da4w	8-bit dynamic activations, 4-bit weights
NPU	Neural Processing Unit
SoC	System on Chip
OTP	One-Time Password
KYC	Know Your Customer
UPI	Unified Payments Interface
RBI	Reserve Bank of India
I4C	Indian Cyber Crime Coordination Centre
NCRP	National Cyber Crime Reporting Portal
SNR	Signal-to-Noise Ratio
WER	Word Error Rate
DGA	Domain Generation Algorithm
ADB	Android Debug Bridge
CI	Continuous Integration
JSON	JavaScript Object Notation
PS3	Problem Statement 3 of the challenge

CHAPTER 1
INTRODUCTION

1.1 Introduction

This chapter introduces the problem of live phone scams in India, explains why the protection must run on the phone itself, and states the objectives and target specifications of KAVACH. It ends with an outline of how the rest of the report is organised.

1.2 Background: Phone Scams in India

Cyber fraud has become one of the largest sources of financial loss for ordinary Indian families. Data shared by the Ministry of Home Affairs in Parliament reports that Indians lost about ₹22,845 crore to cyber fraud in 2024, a 206 % rise over the previous year [1]. “Digital arrest” scams grew especially fast on the National Cyber Crime Reporting Portal: incidents rose from 39,925 in 2022 to 1,23,672 in 2024, and reported losses grew from about ₹91 crore to ₹1,935 crore [2].

What makes these scams hard to stop is that they are conversations, not single messages. The fraudster speaks with authority, creates fear, isolates the victim from family and keeps the call running until the money has been sent. Warnings that appear only after an SMS arrives, or banks that flag a transaction after it is made, are too late for a victim who has already been persuaded on the phone.

1.3 Anatomy of a Phone Scam

Across the twelve scam types in KAVACH’s playbook (Table 1.1), the same sequence of stages appears again and again. KAVACH models this sequence as a kill chain, borrowing the idea from network intrusion analysis [3]: a single suspicious word is weak evidence, but a sequence of stages is strong evidence.

- Contact: an unknown number calls, often claiming to be police, CBI, customs, a bank, a courier company, TRAI or a utility.
- Hook: an authority claim combined with an identity hook or threat (“your Aadhaar is linked to a money-laundering case”, “your SIM will be blocked in two hours”).
- Isolation: the victim is told not to disconnect, not to tell anyone and to stay on a video call.
- Action: the victim is asked for an OTP, to open a link, to install a remote-access app or to share the screen.
- Payment: the victim is told to transfer money, approve a UPI collect request or move savings to a “safe account”.

Table 1.1: Scam types in the KAVACH playbook

Identifier	Scam type
digital_arrest	Digital arrest / fake police
courier_parcel	Courier / parcel with illegal items
kyc_bank	Fake KYC / bank account block
upi_collect_refund	UPI collect request / fake refund
investment_trading	Investment / trading group scam
job_task	Part-time job / task scam
family_emergency	Family emergency / “your son is in trouble”
electricity_bill	Electricity disconnection scam
sim_telecom	TRAI / SIM disconnection scam
lottery_prize	Lottery / prize / KBC scam
remote_support	Fake customer care / remote-access takeover
loan_app	Instant loan app / harassment

1.4 Why the Protection Must Run on the Phone

A system that hears phone calls hears some of the most private moments of a person’s life. Sending that audio to a server, even for a good purpose, creates a store of sensitive recordings that can leak, be misused or be demanded by third parties. It also fails when the network is weak, which is common exactly where many vulnerable users live. KAVACH therefore follows a strict on-device design: the main application does not even request Android’s INTERNET permission, so it cannot send anything off the phone, and audio is processed in memory and never written to storage.

Running everything on a phone is only practical because modern Arm processors can now run compact neural networks in real time. The Arm Bharat AI-SoC Challenge asks exactly this question: what useful AI can run on the Arm system-on-chip in a device people already own? KAVACH answers it for Problem Statement 3, Cybersecurity and Digital Safety, by combining speech recognition, a fast detector and a small large language model on one mid-range phone.

1.5 Problem Statement

Problem Statement 3 of the challenge asks for on-device AI that protects users against scams, phishing and local network threats and explains the risk in plain language. KAVACH addresses it with three on-device pipelines that feed one decision engine:

- Call listening: hears a scam call on speaker, in Hindi, Hinglish or English, and warns during the call.

- Shield: checks messages, links, installed apps, screen sharing and payment-app use, and knows when a call with an unknown number is in progress.
- Network Watch: watches which app talks to which website, flags suspicious look-ups and checks the home Wi-Fi for risky devices.

1.6 Objectives

Main objective: to build an Android application that detects a scam call while it is happening, on one ordinary phone, in Hindi and English, explains the trick with an on-device Llama model, and never sends any data off the phone.

Secondary objectives:

- Recognise Hindi, Hinglish and English speech on the phone, including callers who switch language mid-call.
- Raise CAUTION or STOP early, before the payment step, with zero false STOPs on genuine calls as a hard target.
- Explain every alert in the caller's language with a fine-tuned Llama 3.2 1B model running with ExecuTorch on the Arm CPU.
- Join evidence across channels: a suspicious call followed by a scam SMS, a dangerous link or a remote-access app must escalate.
- Measure speed, memory, battery use and temperature on a real phone, and publish every result and the code that produced it.

1.7 Target Specifications

Table 1.2: Target specifications of KAVACH

Parameter	Target
Platform	One Android phone (Android 10 or later), mid-range Arm SoC
Network use	None: no INTERNET permission in the main app
Audio storage	None: audio kept in memory only for the length of a sentence
Languages	Hindi, Hinglish and English, chosen automatically per sentence
Detector latency	Under 10 ms per sentence on the phone
Speech speed	Faster than real time with both models running
Time to STOP	Median under 30 s from the first scam cue
False STOP rate	At most 2 % on genuine calls (target 0 %)
Explanation	Llama 3.2 1B, 4-bit, on the Arm CPU; never delays the warning
Battery	Under 10 % per hour of listening
User action	Call 1930 (national cyber-crime helpline) offered on every STOP

1.8 Organisation of the Report

- Chapter 2 reviews the background technologies: on-device speech recognition, small language models and their quantisation, LoRA fine-tuning, text classification and Android's rules for call audio.
- Chapter 3 describes the methodology and system design: architecture, one-phone call listening, the speech front-end with Auto language, the detector, the Llama advisor, Shield, Network Watch and the privacy design.
- Chapter 4 describes the implementation: software structure, the Python reference and Kotlin port, continuous integration, data generation, model training and deployment.
- Chapter 5 presents the results: host evaluations, speech experiments, the phone benchmark, replayed and real phone calls, and Llama before and after fine-tuning.
- Chapter 6 discusses limitations, ethics and the threat model, and Chapter 7 concludes and sets out the Phase 2 plan.
- The annexures give installation steps, the repository structure, the Llama output contract, the training configuration and the benchmark fields.

CHAPTER 2
BACKGROUND AND RELATED WORK

2.1 Introduction

KAVACH combines several fields: on-device speech recognition, small large language models and their compression, parameter-efficient fine-tuning, lightweight text classification, and the security rules of the Android operating system. This chapter summarises the background needed to follow the design, and explains where KAVACH differs from existing approaches.

2.2 On-Device Speech Recognition

Modern speech recognisers are neural networks that turn audio features into text. The Conformer architecture [4] combines self-attention, which captures long-range context, with convolution, which captures local patterns in the sound, and is the basis of most current recognisers. Fast Conformer [5] reduces the cost of attention so that long audio can be processed efficiently, which matters on a phone.

Connectionist Temporal Classification (CTC) [6] is a training and decoding method that lets a network output characters or word pieces frame by frame without needing an exact alignment between audio and text. A CTC model is decoded greedily by picking the most likely symbol at each frame and removing repeats and blanks, which is very cheap compared with beam search. KAVACH uses two CTC models:

- AI4Bharat IndicConformer for Hindi: a Conformer model trained on large Indian-language speech corpora such as IndicVoices [7]. It writes Devanagari and also handles English words spoken inside Hindi sentences.
- NVIDIA Parakeet TDT-CTC 110M for English [8]: a Fast Conformer model with 110 million parameters that produces punctuated, cased English.

Both models are run through sherpa-onnx [9], an open-source runtime that executes int8-quantised ONNX models on Arm CPUs, together with the Silero voice-activity detector [10], which decides which parts of the audio contain speech.

2.3 Small Language Models on Phones

Large language models such as the Llama 3 family [11] can read a transcript and explain, in natural language, what kind of scam is happening and what the user should do. Llama 3.2 1B has about 1.2 billion parameters; stored at 16 bits per weight it needs about 2.5 GB, which is too much for comfortable use on a phone. Quantisation stores weights with fewer bits. Groupwise 4-bit quantisation keeps one scale for every small group of weights (32 in KAVACH), which preserves accuracy far better than one scale per row. Methods such as SpinQuant [12] go further by rotating weights before quantisation.

ExecuTorch [13] is PyTorch's runtime for running exported models on phones and embedded devices. Its XNNPACK backend executes the model on the CPU, and on Arm processors it uses Arm KleidiAI micro-kernels [14], which are hand-optimised routines for the matrix multiplications at the heart of a transformer, including 4-bit weight formats. KAVACH exports its fine-tuned Llama to a single .pte file that ExecuTorch loads on the phone.

2.4 Parameter-Efficient Fine-Tuning with LoRA

A base language model does not know KAVACH's output format, the twelve scam types or the rule that every quoted piece of evidence must appear word for word in the transcript. Retraining all 1.2 billion parameters would be slow and expensive. Low-Rank Adaptation (LoRA) [15] freezes the original weights and learns two small matrices for each adapted layer whose product is added to the frozen weight. With rank r = 16 on all attention and feed-forward projections, only about 11 million parameters (0.9 % of the model) are trained, which fits on a free Colab T4 GPU. After training, the LoRA matrices are merged into the base weights, so the exported model has no extra cost at run time.

2.5 Lightweight Text Classification

Not every decision needs a large model. A linear classifier over word and character features can score a sentence in microseconds. Feature hashing [16] maps every feature to one of a fixed number of buckets with a hash function, so the model has a fixed, small size and needs no vocabulary file. KAVACH uses hashed linear classifiers for call windows, SMS text and URLs, and keeps the always-on path free of neural networks so that it is fast, predictable and easy to port exactly between Python and Kotlin.

2.6 Android Rules for Call Audio

Since Android 10, ordinary apps cannot record the audio of a phone call: during a call Android gives the microphone to the call and feeds silence to every other app that records. Android documents one exception for apps that need to keep hearing the user during a call: an app with an enabled accessibility service may continue to capture the microphone [17]. KAVACH uses this exception with a deliberately minimal service that reads no screen content and performs no actions. Because the microphone hears the room, the call must be on speaker for KAVACH to hear the caller.

2.7 Existing Approaches and the Gap

Existing protections fall into a few broad categories. Table 2.1 compares them qualitatively with KAVACH; it describes categories of solution, not specific products.

Table 2.1: Categories of scam protection compared with KAVACH

Approach	Sees the conversation?	Works offline?	Explains in Hindi?
Number-based spam labels	No: only the caller's number	Partly	No
SMS and link filters	No: only text that arrives	Often	Rarely
Cloud call screening	Yes, but audio leaves the phone	No	Varies
Bank-side transaction checks	No: only after payment	No	No
KAVACH	Yes, on the phone, during the call	Yes, fully	Yes, Hindi and English

The gap KAVACH fills is the combination: hearing the conversation itself, during the call, in Indian languages, with an explanation, on a phone people already own, with no audio or text leaving the device.

2.8 Summary

Compact CTC speech models, 4-bit language models running on Arm CPUs through ExecuTorch and KleidiAI, LoRA fine-tuning and hashed linear classifiers together make it possible to understand and explain a phone call on the phone itself. Android's accessibility audio exception makes it possible to do so on the same phone that receives the call. The next chapter shows how KAVACH puts these pieces together.

CHAPTER 3
METHODOLOGY AND SYSTEM DESIGN

3.1 Introduction

This chapter describes how KAVACH is designed. It starts with the overall architecture, then explains how one phone can hear its own call, how speech becomes text in two languages, how the detector decides, how Llama explains the decision, and how the Shield and Network Watch pipelines feed the same decision engine. It ends with the privacy design and the tools used.

3.2 Overall Architecture

KAVACH is one Android application with three on-device pipelines and one decision engine (Figure 3.1). The pipelines produce small, structured events such as “the caller asked for an OTP” or “a dangerous link arrived during an unknown-number call”. The kill chain combines these events over time and across channels and decides the warning level. Llama is called only when the level rises, so it never slows down the always-on path.

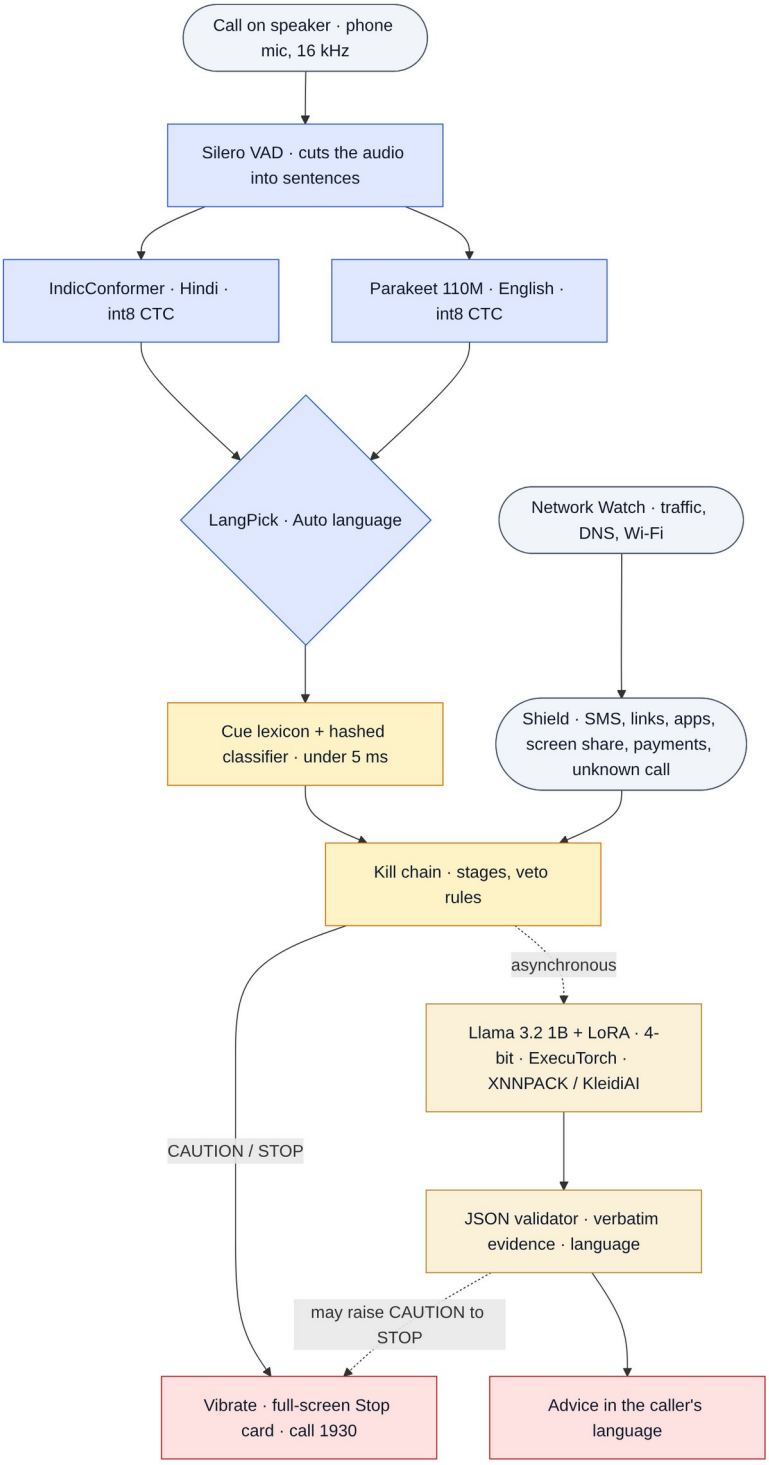


Figure 3.1: Overall architecture of KAVACH: three on-device pipelines feed one kill chain; Llama explains alerts

- Pipeline A, Call listening: microphone → voice-activity detection → Hindi and English speech recognition → Auto language choice → cue lexicon and classifier.
- Pipeline B, Shield: message notifications, links, installed apps, screen sharing, payment-app use and call state.

- Pipeline C, Network Watch: per-app traffic counters inside Shield, plus an optional DNS-watch and home Wi-Fi app.
- Decision: the kill chain combines evidence from all three and raises INFO, CAUTION or STOP.
- Explanation: Llama 3.2 1B reads the recent transcript and evidence and returns a JSON verdict with advice in the caller's language.

3.3 Listening to a Call on the Same Phone

During a phone call Android routes the microphone to the call and gives silence to every other recording app. The documented exception is an app with an enabled accessibility service. KAVACH ships CallListenAccessService for this purpose only: it asks for no screen content, performs no gestures and reads no other app. While it is enabled and the user has turned on "Protect my calls", KAVACH starts its microphone service when a call from a number not in the contacts begins (Figure 3.2).

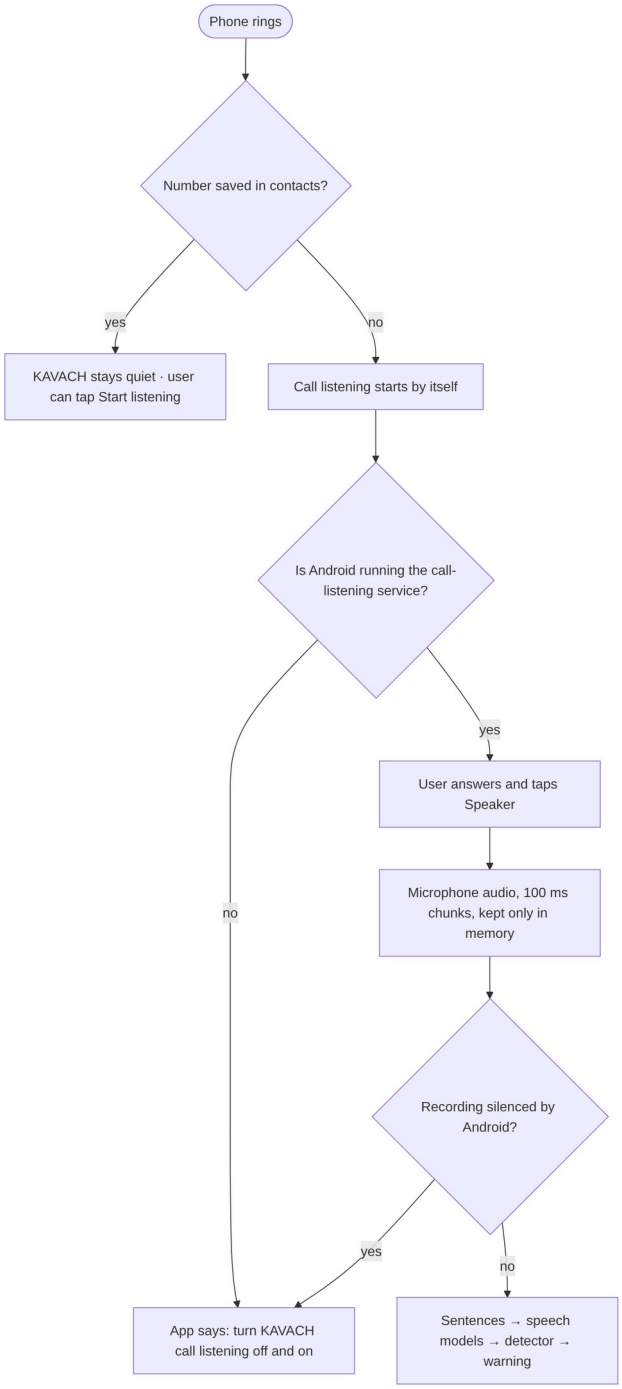


Figure 3.2: Flow of one protected call from an unknown number

Three design choices make this work in practice:

- Speaker mode: the microphone hears the room, so the caller is heard only when the call is on speaker. The KAVACH Phone dialer (optional) turns the speaker on by itself for unknown numbers.
- Silent warnings during a call: a spoken warning would be sent into the call and would also block KAVACH's own listening, so during a real call the phone vibrates and shows the full-screen STOP card instead.

- Health checks: if Android stops the app (for example after the default dialer is changed), it can mark the accessibility service as crashed while its switch still shows “on”. KAVACH checks that the service is really running and whether its recording is being silenced during a call, and shows a Fix button when either goes wrong.

3.4 Speech Front-End

Audio is captured at 16 kHz mono in 100 ms chunks with automatic gain control and noise suppression turned off. Silero VAD marks where speech begins and ends, and each complete sentence is handed to the recognisers. Audio exists only in a memory buffer for the length of one sentence and is never written to storage (Table 3.1).

Table 3.1: Stages of the speech front-end

Stage	Implementation	Notes
Capture	AudioRecord, VOICE_RECOGNITION source, 16 kHz mono, 100 ms chunks	No audio written to storage
Voice activity	Silero VAD through sherpa-onnx, 512-sample windows	Cuts audio into sentences
Hindi / Hinglish	AI4Bharat IndicConformer Hindi, 120 M parameters, int8 CTC, 198 MB	Writes Devanagari; handles English words in Hindi
English	NVIDIA Parakeet TDT-CTC 110M, int8 CTC, 126 MB	Punctuated, cased English
Fallback	Small streaming Zipformer models (Hindi 69 MB, English 188 MB)	Used only if the big models are absent
Placement check	SNR estimate from 20 ms frame powers over 5 s	“Good / move closer / too noisy” on screen
Own-voice filter	Sentences heard while KAVACH speaks, plus 1.2 s, are ignored	Stops KAVACH warning itself

3.4.1 Auto Language Choice

Indian callers often begin in English and switch to Hindi or Hinglish, or the other way round. Asking the user to choose a language when an unknown number calls is unrealistic, so the default call language is Auto. In Auto mode both recognisers decode every sentence, and LangPick keeps the transcript that looks more like real words of its own language. Each model turns speech of the other language into words that are not real words of its own language, so for each transcript LangPick measures k, the share of its words found among the 20,000 most frequent words of that language, and c, the share found among the 200 most frequent (mostly function words such as “the” or “है”). Word lists come from wordfreq [18]. Shares are counted over n + 1 words, so a one- or two-word fragment cannot win on its own. The score is:

$$score = (k_en - k_hi) + W \times (c_en - c_hi), \quad W = 1.0$$

English is chosen when the score is above a margin of 0.2; otherwise Hindi/Hinglish is chosen, which is the safe default because the Hindi model also writes English loan words that the lexicon knows in Devanagari. A sentence whose score lies within 0.15 of the margin (short words such as “okay” or “haan”) keeps the previous sentence’s language. Warnings and Llama’s explanation follow the majority language of the last three sentences. The Python version (kavach_core/langpick.py) and the Kotlin version (LangPick.kt) are checked against the same golden vectors.

3.5 The Always-On Detector

The detector runs after every recognised sentence and at least every three seconds. It has three layers, all simple enough to run in about 4 ms on the phone and to be ported exactly between Python and Kotlin.

3.5.1 Cue Lexicon

The cue lexicon (assets/cues.json) holds 698 phrases in 16 categories, each written in English, romanised Hinglish and Devanagari (Table 3.2). Phrase rules allow prefixes and gaps of up to four words, so “share the six digit OTP” still matches “share OTP”. A request is ignored when a nearby word negates it, so a bank’s “never share your OTP” counts as safety advice, not as a request.

Table 3.2: Cue categories of the lexicon

Category	Examples of what it catches
authority_police	police, CBI, customs, cyber cell, TRAI officer
authority_bank	RBI officer, bank manager, KYC department
identity_misuse	your Aadhaar / PAN / SIM is linked to a case
threat	arrest warrant, account blocked, legal action, FIR
urgency	within two hours, immediately, right now
isolation	do not tell anyone, do not disconnect, stay on video
credential_request	tell me the OTP, PIN, CVV, card number
remote_access	install AnyDesk, share your screen
link_click	click the link, open the link I sent
payment_request	transfer to safe account, pay the fee, scan this QR
reward_greed	you have won, lottery, cashback, high returns
family_emergency	your son has been arrested, accident, hospital
safety_advice	never share your OTP, the bank never asks
victim_sharing	the user reading out an OTP or account number
victim_paying	“I am transferring now”, “main transfer kar raha hoon”
victim_fear	“please don’t arrest me”, “mujhe arrest mat karo”

3.5.2 Hashed Linear Classifier

A logistic-regression classifier scores the last 120 words of the call. Its features are FNV-1a hashes of words, word pairs and three- to five-letter fragments, plus the cue categories that fired and pairs of categories. Hashing gives a fixed model size of about 1 MB with no vocabulary file. Every score can be broken down into per-word contributions, which become the evidence words shown to the user. The same design is used for SMS text and URLs.

3.5.3 Kill Chain

The kill chain keeps the evidence from all channels. Evidence fades with a half-life that depends on the channel: 10 minutes for calls, 30 minutes for messages and links, and 24 hours for apps. The level (none, info, caution, stop) is sticky within a session, so a scammer cannot calm the guard with small talk. Table 3.3 lists the rules that raise STOP. The classifier adds CAUTION or STOP when it stays above 0.8 for two ticks in a row, and can veto a weak rule-only STOP while it is confidently benign (below 0.5); strong rules are never vetoed.

Table 3.3: Rules of the kill chain

Rule	Fires when	Level
isolation + authority	an authority claim and an isolation demand	STOP (vetoable)
authority + identity + threat	all three heard	STOP
multi-action	two different action or payment requests	STOP
hook + action	a hook and an action with no safety advice heard	STOP
victim complying	user reads an OTP or says they are paying, and a scam pretext is present	STOP
call + action	a suspicious call and a risky action in the same call	STOP
unknown call + risky action	dangerous link, APK, screen share or upload during an unknown-number call	STOP
cross-channel	evidence from two channels within the half-life	STOP
total score ≥ 7	weighted evidence total	STOP
total score ≥ 3	weighted evidence total	CAUTION
payment app in long call	payment app opened during an unknown call longer than 3 min	CAUTION

Figure 3.3 shows how the level moves through the stages of a scam, and Figure 3.4 shows the order of events from a spoken sentence to the warning and the explanation.

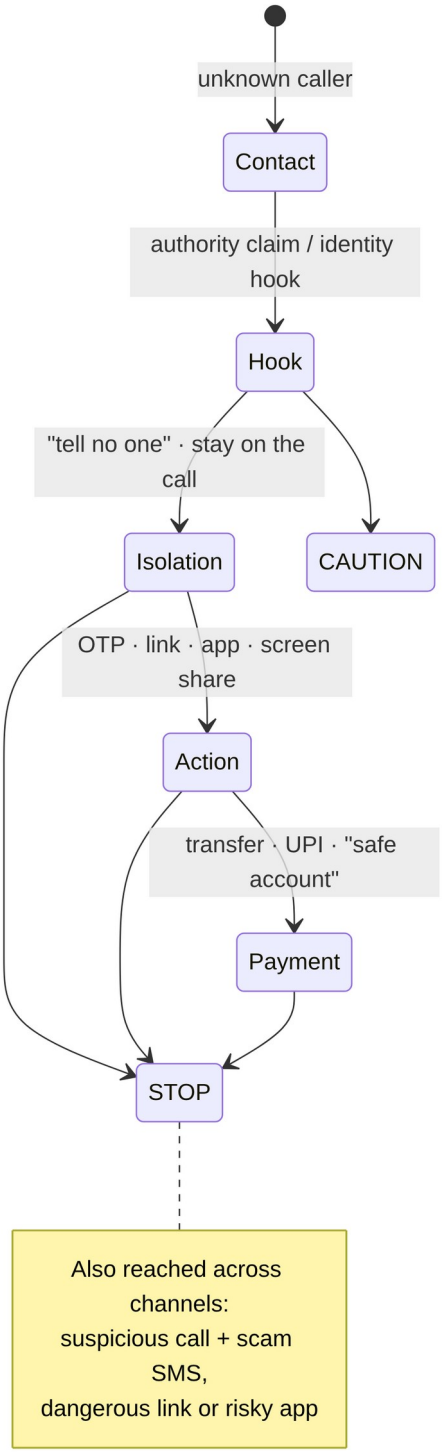


Figure 3.3: States of the kill chain from first contact to STOP

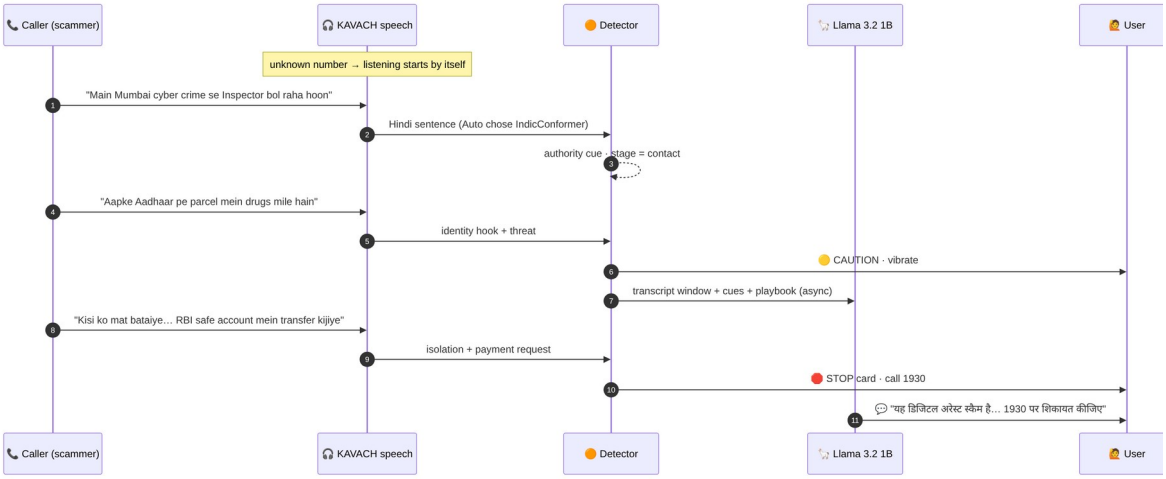


Figure 3.4: Sequence from a spoken sentence to the warning and Llama's explanation

3.6 The Llama Advisor

Rules can say that a call is dangerous, but not always why in words a frightened person understands. KAVACH therefore adds Llama 3.2 1B Instruct, fine-tuned with LoRA, as an advisor. It runs only when the level rises to CAUTION or STOP, on a dedicated thread, and the rule-based warning is always shown first.

3.6.1 Prompt and Output Contract

The prompt contains a constant system part (the rules of the contract and the twelve scam types from the playbook) and a short variable part (the last part of the transcript, the cue evidence and the call language). The constant part is pre-filled into the KV cache when the app starts and re-armed after every alert (the "warm prefix"), which halves the time to the first token. The model must return exactly one JSON object (Table 3.4).

Table 3.4: Fields of Llama's JSON verdict

Field	Type	Meaning
scam	true / false	Is this a scam?
type	one of 12 identifiers or none	Scam type from Table 1.1
stage	none / contact / hook / isolation / action / payment	How far the scam has gone
confidence	0.0 to 1.0	Model's confidence, clamped by the validator
evidence	up to 3 quotes	Exact words from the transcript
say	up to 35 words	Advice to the user in the call's language
action	none / continue_with_care / verify_independently / do_not_share / do_not_pay / hang_up	Recommended action

3.6.2 Validator

Language models can make things up, so every answer passes through a validator before the user sees it:

- Enumerated fields must hold allowed values; “scam”: “maybe” fails validation (a real bug found by the unit tests).
- Confidence is clamped to the range 0 to 1.
- Every evidence quote must appear word for word in the transcript; quotes that do not are dropped and the confidence is capped.
- The advice is cut at the last complete sentence, so a half-finished sentence is never shown.
- Advice that copies the example in the prompt is replaced by the vetted playbook text and labelled as such; a copied answer can never raise the warning level.
- On invalid output the model is asked once more; if it fails again the playbook text is used.

Llama may raise CAUTION to STOP when its self-reported confidence is at least 0.8 and the stage is isolation or later. This confidence is the model’s own number, not a calibrated probability. Llama can never lower or clear any warning: an invalid, timed-out or missing answer leaves the rule-based warning exactly as it was. This also limits the damage a caller could do by speaking instructions aimed at the model.

3.6.3 LoRA Fine-Tuning

Table 3.5: LoRA fine-tuning configuration

Setting	Value
Base model	meta-llama/Llama-3.2-1B-Instruct
Adapter	LoRA, rank r = 16, alpha 32, dropout 0.05
Adapted layers	q, k, v, o, gate, up and down projections
Training data	4,000 instruction pairs (dev 400) built by the same prompt code the phone uses
Epochs and batch	1 epoch, batch 2 × 8 accumulation steps (16), learning rate 2 × 10^{-4}, cosine schedule, 3 % warm-up
Hardware	Free Google Colab T4 GPU, fp16, 4.1 hours
Final losses	training 0.111, dev 0.041
After training	LoRA merged into the base weights before export

3.6.4 Export and Quantisation

The merged model is exported with ExecuTorch 1.4 to a single .pte file using 8da4w quantisation: 4-bit weights in groups of 32 with 8-bit dynamic activations, and 4-bit embeddings. The file is 820 MB, compared with about 2.5 GB at 16 bits. On the phone it runs

on the Arm CPU through the XNNPACK backend, which uses Arm KleidiAI micro-kernels for the 4-bit matrix multiplications. Decoding is greedy and stops as soon as the JSON object closes. Each generation is limited to 35 seconds, a value set from the phone measurements (about 15 s on the warm path and about 26 s on the cold path of the fine-tuned file). The timer covers waiting for a background prefix prefill, the prompt prefill and the generation, but not loading the model. Earlier versions of this report wrongly gave 8 seconds.

3.7 Shield

- Message Shield reads SMS, WhatsApp and Telegram notifications (so it needs no READ_SMS permission) and runs the SMS classifier, the cues and Link Check on every message.
- Link Check tests an offline allow-list of genuine Indian domains, including the RBI-mandated .bank.in and .fin.in suffixes [19]; then look-alike rules (brand in an unofficial domain, typosquatting by edit distance, homoglyphs and punycode, shorteners, raw IP addresses, the @ trick, APK downloads, scam-prone top-level domains, brand pages on free website builders); then the URL classifier with a threshold set for at most 3 % false alarms.
- App Inspector (preview) scores each new app’s permissions: SMS, accessibility and overlay together, sideloading, and names that imitate a bank or KYC app.
- Screen-share Guard (preview) raises STOP when a payment or banking app comes to the front while a remote-access app is active.
- Call awareness knows when a call is on, whether the number is a contact and how long the call has lasted, without hearing the call.
- Every STOP opens a full-screen card with a plain reason and a one-tap call to 1930; the user may also choose to alert a family member by SMS.

3.8 Network Watch

- App traffic watch (inside the main app, with no INTERNET permission): NetworkStatsManager byte counters per app every 30 seconds. Only suspect apps (sideloaded, new, remote-access or rated suspicious) are judged. Kinds of finding: suspicious upload (at least 5 MB in 5 minutes in the background, or a slow trickle of 4 MB per hour) and beaconing (small background bursts at a steady rhythm).
- DNS watch (separate optional app): a local DNS-only VPN learns which app looks up which name and forwards the look-up unchanged. It flags bank look-alikes and bursts of machine-generated names that do not exist, which is how malware searches for its control server.
- Home Wi-Fi check: tries ten commonly abused services (ADB 5555, Telnet, SMB, RDP, TR-069, FTP, RTSP, SSH, HTTP) on the local network and explains in plain language how to fix any it finds.

- Flow model: a 1,665-parameter network trained on UNSW-NB15 [20], reported for research but not used for alerts (Chapter 5 explains why).

3.9 User Interface

The app was designed for users who may not be comfortable with technology. Figure 3.5 shows the main screens: the home screen with the protection status, the call-language control (Auto | हिंदी | English), the live call screen with the transcript and level, and the full-screen STOP card with the reason and a one-tap call to 1930.

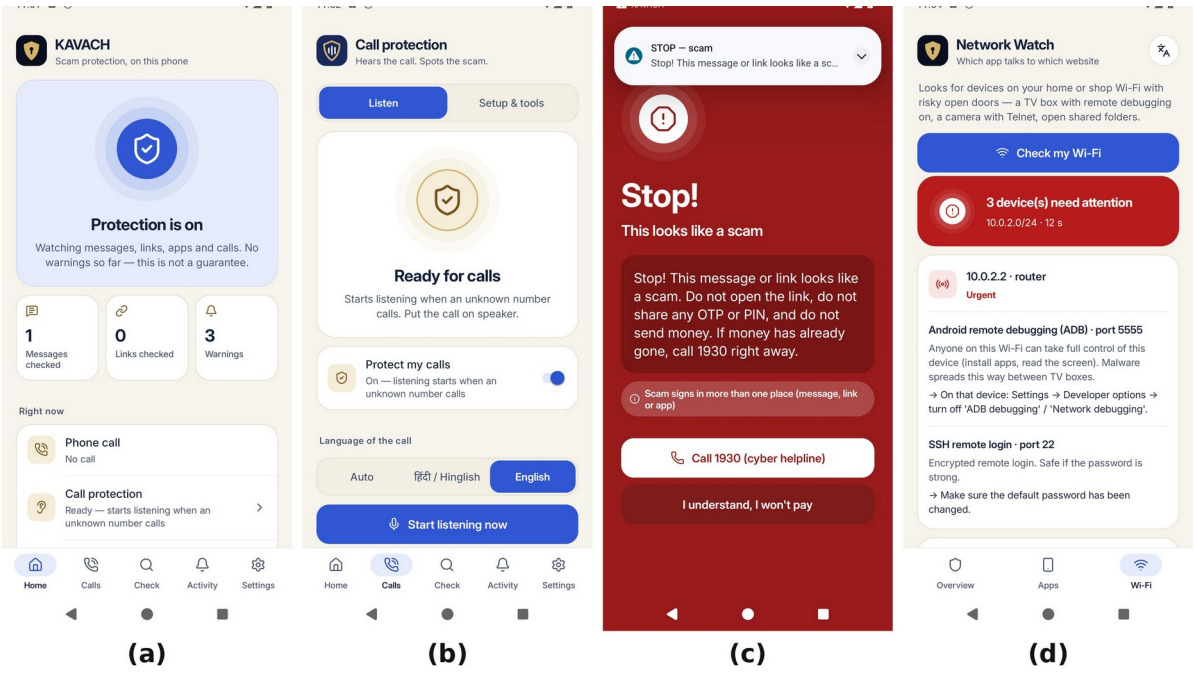


Figure 3.5: Main screens of the KAVACH app

3.10 Privacy and Security Design

Table 3.6: Privacy measures and how they are enforced

Measure	How it is enforced
No upload from the main app	The main app's manifest has no INTERNET permission, so speech, detection and Llama output cannot be sent by it
Optional paths, user-enabled	An SMS warning to one family member; a Bluetooth link to a paired KAVACH phone (encrypted, paired devices only); the separate Network Watch app forwards DNS look-ups unchanged
Audio never stored	Audio is held only in a memory buffer for one sentence; no file is ever written
Transcript kept briefly	Transcript lives in memory for the session; logs keep only levels and timing
No SMS permission	Messages are read from notifications the user allowed
Network content not read	Traffic watch sees only byte counts per app; DNS watch sees only names, never contents

Measure	How it is enforced
Minimal accessibility use	The call-listening service reads no screen content and performs no actions
User consent	Every permission is explained on screen before Android asks for it

3.11 Tools and Technologies

Table 3.7: Tools and technologies used

Area	Tools
Android app	Kotlin, Android SDK 34 (installs on Android 8+; call listening needs Android 10+), Gradle
Speech	sherpa-onnx 1.13.8, Silero VAD, IndicConformer, Parakeet, Piper TTS (testing)
Language model	Llama 3.2 1B Instruct, PyTorch, Transformers, PEFT, ExecuTorch 1.4, XNNPACK, KleidiAI
Classifiers	Python 3, NumPy, scikit-learn; Kotlin port
Training	Google Colab (T4 GPU), Hugging Face Hub
Testing and CI	pytest, JUnit, GitHub Actions, ADB
Analysis	matplotlib, pandas, Scapy

3.12 Summary

KAVACH listens through Android's accessibility audio exception, recognises each sentence in Hindi and English and keeps the better transcript, scores it with a fast and explainable detector, combines it with evidence from messages, links, apps and the network, and asks a fine-tuned 4-bit Llama to explain the result in the caller's language. Every part runs on the phone, and nothing is stored or sent.

CHAPTER 4
IMPLEMENTATION

4.1 Introduction

This chapter describes how the design of Chapter 3 was built: the structure of the code, how the Python reference and the Kotlin app are kept identical, how the training data was produced, how Llama was trained and exported, and how the models are installed on the phone.

4.2 Software Structure

Table 4.1: Main parts of the repository

Folder	Contents
kavach_core/	Python reference: lexicon, classifiers, kill chain, LangPick, prompts and JSON validator
android/core/	Kotlin port of kavach_core with parity tests
android/guardian/	The KAVACH app: call listening, speech, Shield, Llama, screens
android/netwatch/	Optional Network Watch app (DNS watch, Wi-Fi check)
assets/	Single source of truth: cues, playbook, domain lists, model weights, word lists
data/	Generators for synthetic calls, SMS and hand-written sets (raw data never committed)
models/	Training scripts for classifiers, ASR noise data, Llama LoRA and export
bench/	Evaluation and benchmark scripts and their saved results
notebooks/	Colab notebook for LoRA training and export
tests/	35 pytest tests and the parity-vector generator
docs/	Technical report, benchmarks, audit, model and data cards

4.3 Android Application

The app is a single Gradle module (:guardian, application id io.kavach.guardian) that depends on the shared :core module. Its main components are:

- GuardianService: the foreground microphone service; runs VAD, both recognisers and the detector, writes a performance row every five seconds and raises warnings.
- AsrEngine: builds the sherpa-onnx recognisers (sentence mode for IndicConformer and Parakeet, streaming fallback) and runs LangPick in Auto mode.
- CallListenAccessService and CallMonitor: the accessibility audio exception and call-state awareness.
- ShieldHub and KillChain: collect events from every channel and decide the level.

- LlmAdvisor: loads the .pte once with ExecuTorch's LlmModule, keeps the warm prefix and validates the JSON.
- Screens: Home, Calls (with the language control and readiness rows), live call, STOP card, Benchmark and Setup & tools.

4.4 Keeping Python and Kotlin Identical

Every rule and model is first written and evaluated in Python, then ported to Kotlin. To make sure the phone behaves exactly like the evaluated code, tests/make_parity_vectors.py writes golden outputs from Python and the Kotlin tests check 336 of them: text normalisation, hashing, lexicon hits, feature vectors, the logits of all three classifiers, URL and message verdicts, kill-chain steps, prompts, JSON validation, full Guardian replays, Network Watch traces, DNS parsing, the placement SNR and LangPick choices. Porting found and fixed two real bugs: Python's brand ordering depended on hash randomisation, and the Java and Python IDNA decoders disagreed on some punycode.

4.5 Continuous Integration

GitHub Actions runs the Kotlin parity tests and builds both debug APKs on every push; the 35 Python tests are run locally before each commit. A separate workflow mirrors the large speech model files so the phone set-up script can fetch them from one place. Debug APKs are built for both apps.

4.6 Training Data

No real victim audio or personal data is used. Call transcripts are generated by data/synth/generate_calls.py from 11 scam scripts and 9 genuine scenarios, built from 47 utterance pools paraphrased from I4C, RBI and bank advisories. Each pool has English, Hinglish and Devanagari versions, and the generator adds ASR-style errors and realistic timing (Table 4.2).

Table 4.2: Data used for training and evaluation

Set	Source	Size / use
Synthetic calls	Generator; template-disjoint splits	train 8,000 / dev 600 / test 1,000
Caller through speaker	Test calls with 3× ASR errors on caller lines	1,000; reported only
Hard negatives	Genuine bank alerts, delivery OTPs, relatives asking money, people discussing scams	in every split
Hand-written calls	Written by a different author style	30; reported only
Locked held-out calls	Written before any tuning	20; reported once
ASR-noisy calls	Piper TTS [23] → speaker channel → recogniser	88 English, 410 Hindi

Set	Source	Size / use
SMS	UCI SMS Spam [21] + 4,300 synthetic Indian messages	sms_model
URLs	Phishing.Database, popular domains, Indian look-alikes	url_model
Network	UNSW-NB15 [20], NSL-KDD [22], 25 DGA families	Network Watch
Llama	Instruction pairs from the phone's prompt code	4,000 / 400

Template-disjoint splits are essential. Each utterance pool is divided by item into train, dev and test, so dev and test contain phrasings never seen in training. With an earlier split that shared templates, the call classifier looked perfect on dev (F1 0.999) but scored only 0.817 on test.

4.7 Training and Exporting Llama

The Colab notebook notebooks/kavach_llama_lora_colab.ipynb runs the whole Llama pipeline on a free T4 GPU, with checks after every step:

1. Check the GPU and mount Google Drive for a safety copy.
2. Clone the repository and install the pinned packages (torchao is removed because it conflicts with PEFT).
3. Log in to Hugging Face with a read token (Llama 3.2 is a gated model).
4. Build the 4,000 / 400 instruction pairs with models/llama/make_sft_data.py.
5. Score the base model on 100 dev examples with models/llama/eval_llama.py.
6. Train the LoRA adapter (finetune_lora.py) and save it to Drive.
7. Score the fine-tuned model on the same 100 examples.
8. Merge the adapter and export the 8da4w .pte with ExecuTorch 1.4.
9. Copy the .pte, the scores and the logs to Drive.

4.8 Deployment to the Phone

The APK does not contain the large models. scripts/push_models.sh copies them to the app's private folder over ADB: the VAD, IndicConformer, Parakeet, the small fallback models, the Llama .pte and its tokenizer. The app checks each file on start and shows which ones are ready on the Readiness screen. The Benchmark screen measures speech speed, Llama speed, battery and temperature and writes a JSON file that is pulled back with adb pull.

4.9 Summary

The implementation keeps one source of truth for every rule and model, proves with 336 parity checks that the phone runs the same logic that was evaluated, uses only synthetic and public data, and makes the whole Llama pipeline reproducible on a free GPU.

CHAPTER 5
RESULTS AND DISCUSSION

5.1 Introduction

This chapter reports every measurement made in Phase 1, in the order a call is processed: the text classifiers, call-level detection, speech in noisy rooms, Hindi speech, Auto language, tests on the phone with replayed audio and with real calls, the Llama advisor before and after fine-tuning, the phone benchmark and Network Watch. Every number comes from a script and a saved result file in the repository; nothing is estimated by hand. Results that are weak or not yet measured are stated as such.

5.2 Evaluation Sets and Metrics

Table 5.1: Metrics used in this chapter

Metric	Definition
STOP recall	Share of scam calls that reach STOP at any point
≥ CAUTION	Share of scam calls that reach CAUTION or STOP
STOP before money	Share of scam calls that reach STOP before the first payment request
Time to STOP	Seconds from the first scam cue to STOP (median)
False STOP / CAUTION	Share of genuine calls that reach STOP / CAUTION
Precision, recall, F1	Standard definitions on the positive (scam) class
WER	Word error rate of a transcript against the script
RTF	Processing time divided by audio length; below 1 is faster than real time
TTFT, tok/s	Llama time to first token, and tokens generated per second

The call sets are described in Table 4.2. Thresholds were tuned on the development sets only. The hand-written and locked held-out sets were run once for reporting and never used for tuning.

5.3 Text Classifiers

Table 5.2: Text classifiers on held-out test splits

Model	Task	Test n	Precision	Recall	F1	AUC
cue_model	120-word call window → scam	9,010	0.946	0.883	0.913	0.962
sms_model	SMS / notification → scam	1,914	0.994	0.901	0.945	0.990
url_model	URL → phishing (rules + model)	10,196	0.987	0.752	0.853	0.945

All three classifiers have high precision, which matters because a false alarm on a genuine message teaches the user to ignore KAVACH. Adding the cue categories as features raised the call classifier's test F1 from 0.817 to 0.903, and the three victim-side categories raised it to 0.907. Link Check flags 98.5 % of generated Indian look-alike domains and 0 % of genuine Indian domains; it flags 3.9 % of popular global domains and 69 % of generic global phishing domains, many of which are random strings with no brand in them and stay "unknown" at the chosen 3 % false-alarm budget.

5.4 Call-Level Detection

Each call is replayed in call time through the full detector, exactly as the phone would see it sentence by sentence. Table 5.3 compares the rules alone with the shipped detector (rules, classifier and veto). All sets were re-run on 12 November 2026 with the shipped models; earlier builds had reported 90.6 %, 71.1 % and 7 / 10 on the first, second and fourth sets.

Table 5.3: Call-level results (rules only vs shipped detector)

Set (scam / genuine)	Config	STOP	≥ CAUTION	STOP before money	Median time to STOP	False STOP	False CAUTION
Synthetic test (511 / 489)	rules	423 (82.8 %)	472 (92.4 %)	336 (65.8 %)	17.3 s	0	0
	shipped	477 (93.4 %)	508 (99.4 %)	420 (82.2 %)	12.6 s	0 / 489	29 (5.9 %)
Caller through speaker (502 / 498)	rules	312 (62.2 %)	360 (71.7 %)	218 (43.4 %)	18.9 s	0	0
	shipped	375 (74.7 %)	466 (92.8 %)	304 (60.6 %)	14.6 s	0 / 498	26 (5.2 %)
Hand-written (16 / 14)	rules	12 / 16	14 / 16	7 / 16	24.7 s	2 / 14	3 / 14
	shipped	15 / 16	15 / 16	10 / 16	19.0 s	0 / 14	3 / 14
Locked held-out (10 / 10)	rules	5 / 10	8 / 10	2 / 10	21.2 s	0 / 10	1 / 10
	shipped	8 / 10	8 / 10	5 / 10	21.3 s	0 / 10	4 / 10

Three observations follow. First, the shipped detector raises STOP recall by 10 to 30 points on every set, and no genuine call reached STOP in any set; the classifier veto removed both rule-only false STOPs on the hand-written set (people discussing a scam, and an office call about screen sharing and payments). Second, the median time to STOP is below 30 s on every set. Zero false STOPs does not prove the 2 % target, however: on the 24 human-written genuine calls the 95 % Wilson upper bound is about 14 %, and on the 10 held-out genuine calls about 28 %. Third, the target of catching 90 % of scams before the payment step is not met (50 % to 82 %), and false CAUTIONs on the small human-written sets are high (3 of 14 and 4 of 10). Llama

cannot reduce these CAUTIONs, because it is only allowed to escalate a warning, never to clear one; reducing them needs better detection, not a more confident explanation. The held-out set has been scored on several builds and is no longer pristine.

5.5 Does Cross-Channel Fusion Help?

Joining evidence across channels is KAVACH's main new idea, and the rules-versus-shipped comparison above does not test it. The script bench/fusion_ablation.py therefore replays identical timestamped event traces under five configurations: A, the call detector alone; B, the message, link and app detectors alone, with no knowledge of the call; C, A and B combined independently (the higher level wins, with no correlation); B+, Shield with call awareness but no listening; and D, full fusion exactly as wired in the app. Configuration E (D plus Llama escalation) needs the deployed model and was not run on the host. Calls come from the caller-through-speaker test set; the SMS texts are 8 scam and 4 genuine team-written examples; each template is run on 40 sessions; all thresholds are the shipped ones. The plan was written into the script before the first run (Tables 5.4 and 5.5).

Table 5.4: Fusion ablation, scam sessions: STOPs out of 40 per template

Scam template	A	C	B+	D	Warned before the risky step (C / D)
Scam call alone	37	37	0	37	31 / 31
+ scam SMS with a link mid-call	32	40	40	40	38 / 38
+ APK install	30	30	40	40	32 / 32
+ remote access, then a payment app	36	36	40	40	29 / 29
Scam SMS 5 min before the call	30	37	33	40	37 / 37
First 40 % of the call, then an APK	7	7	40	40	26 / 26

Table 5.5: Fusion ablation, genuine sessions: STOPs out of 40 (in brackets: STOPs that blame the call)

Genuine template	C	B+	D	D ≥ CAUTION
Genuine call alone	0	0	0	5
+ unrelated scam SMS mid-call	32 (0)	32 (22)	40 (32)	40
+ scam SMS 20 min after the call	32 (0)	32 (0)	32 (3)	40
+ payment app opened during the call	0	0	4 (4)	4
+ risky APK installed 2 days earlier	0	0	0	2
+ genuine bank SMS, duplicated and reordered	0	0	0	3
+ screen sharing for work, no payment	0	40 (40)	40 (40)	40

Fusion turns an ambiguous call into STOP at the risky action: an APK install during the call rises from 30 to 40 of 40, remote access from 36 to 40, and a call heard only in part from 7 to 40. It does not warn earlier: the share of scams warned before the risky step is identical for C and D in every template. Most of the gain comes from knowing that an unknown-number call is in progress (B+), not from what was heard; speech adds 3 of 40 only when a scam SMS arrived five minutes before the call. The costs are real: an unrelated scam SMS during a genuine call is blamed on the call in 32 of 40 sessions, screen sharing for work during an unknown-number call is STOP by design, and a payment app opened during a genuine call is STOP in 4 of 40. The rules join events by time only; matching the organisation named on the call with the sender of a message is planned for Phase 2. These are synthetic traces replayed on the host, so replay timing says nothing about live Android latency.

5.6 Speech in Noisy Rooms

Four scripted calls (two scam, two genuine) were spoken with Piper TTS, passed through a telephone channel and room reverberation, and mixed with fan or traffic hum, a television, or several people talking, at 30, 15, 8 and 3 dB signal-to-noise ratio: 48 noisy calls per configuration (Table 5.6).

Table 5.6: English recogniser choices in noisy rooms (host)

Recogniser + front-end	Scam STOP at 30 / 15 / 8 / 3 dB	Genuine STOP	WER hum / TV / babble	RTF
Small 20M model	3/6 · 3/6 · 0/6 · 0/6	0/24	0.53 / 0.80 / 0.74	0.05
20M + GTCRN denoiser	1/24 in total	0/24	0.48 / 0.97 / 1.13	+0.08
20M + scam-word hotwords	3/24 in total	0/24	0.60 / 0.74 / 0.75	–
Zipformer en-2023-06-21	6/6 · 6/6 · 4/6 · 2/6	0/24	0.20 / 0.83 / 0.90	0.12
Zipformer + hotwords	6/6 · 6/6 · 5/6 · 2/6	0/24	0.19 / 0.85 / 0.86	0.12

The bigger model let three times more noisy scam calls reach STOP, so it replaced the small one (it is now the fallback behind Parakeet). The denoiser was rejected: it helps on steady hum but invents speech-like fragments from TV and babble, pushing WER above 1. Hotwords added one call and were not worth the cost. With a TV or other people talking, even the bigger model transcribes the background voices, which is why the app shows a placement check. Its SNR estimate reads about 5 dB high, and its “good” threshold of 18 dB was set from these runs. With one TTS realisation per condition these numbers show a direction, not precise rates.

5.7 Hindi Speech Recognition

Hindi was the weakest part of the first build. One hundred unseen synthetic Hindi calls (50 scam, 50 genuine) were spoken by Piper TTS and decoded first by the small streaming Hindi

model, before and after lexicon and classifier improvements, and then by IndicConformer on the same audio (Tables 5.7 and 5.8, Figure 5.1).

Table 5.7: Streaming Hindi model before and after lexicon and classifier fixes

Condition	Build	Scam → red	Scam → yellow or red	Genuine → red
Near speaker, quiet room	before	24 %	58 %	0 %
Near speaker, quiet room	after	44 %	88 %	0 %
Telephone band, SNR 10–30 dB	before	0 %	12 %	0 %
Telephone band, SNR 10–30 dB	after	0 %	44 %	0 %

Table 5.8: Streaming Hindi model vs IndicConformer on the same 100 calls

Condition	Streaming: red / yellow-or-red	IndicConformer: red / yellow-or-red	Genuine → red / yellow (IndicConformer)
Near speaker, quiet room	44 % / 92 %	66 % / 96 %	0 % / 6 %
Telephone band, SNR 10–30 dB	0 % / 46 %	58 % / 86 %	0 % / 6 %

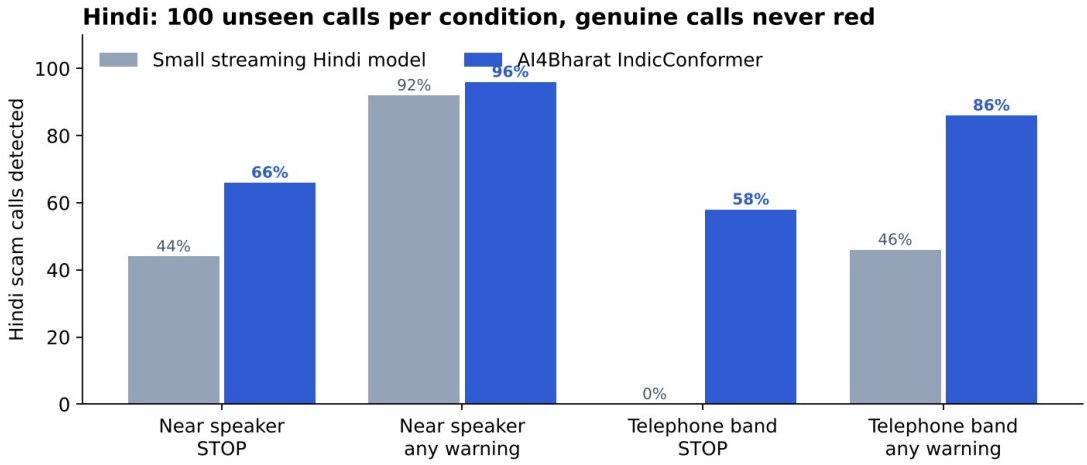


Figure 5.1: Hindi scam calls reaching STOP: streaming model vs IndicConformer

On telephone-band audio, the condition closest to a real call, STOP recall rose from 0 % to 58 % with no false STOP. The three genuine calls that reached yellow are the closest benign twins of scams: a real traffic-police call and two real card-fraud checks from a bank. Lexicon fixes alone could not solve the problem because the small model's text was mostly unusable in noise; a better recogniser was the real fix.

5.8 Auto Language Choice

LangPick was evaluated per sentence on the held-out test split, using the first two caller sentences of each call (Table 5.9).

Table 5.9: Per-sentence language choice in Auto mode

Channel	English chosen for English speech	Hindi chosen for Hindi speech
Telephone band, SNR 10–30 dB	93.0 % (80 / 86)	91.0 % (91 / 100)
Near speaker	87.2 % (75 / 86)	99.0 % (99 / 100)
Overall	–	92.7 % (345 / 372)

Most errors are short English sentences whose Hindi transcript is a one- or two-word fragment. The stickiness rule and the three-sentence majority for warnings limit the effect of a single wrong choice. How much a wrong choice changes the final warning is not yet measured.

5.9 Tests on the Phone with Replayed Audio

The team’s six test files were played from a laptop speaker 10 to 30 cm from the phone, with IndicConformer and Parakeet installed and the call language on Auto (Table 5.10).

Table 5.10: Replayed test files on the phone (12 November 2026)

File	Expected	Result
Hindi digital arrest	STOP	STOP (twice)
Hindi genuine bank call	no warning	lowest hint only (“worth watching”, one misheard word)
English digital arrest	STOP	STOP
English fake KYC	STOP	STOP
English delivery call	no warning	no warning
English genuine bank call	no warning	no warning

In the first phone build, two English scam files stopped at yellow. Investigation showed that the spoken yellow warning muted the microphone for 19 to 33 seconds and the decisive sentences were lost. Yellow warnings are now silent (vibration only), so the microphone is never muted before the decision, and both files reach STOP.

5.10 Real Phone Calls

A teammate called the test phone over the normal cellular network from a number not in its contacts. The call was answered in the phone’s own dialer and put on speaker; KAVACH

started listening by itself with the call language on Auto. The teammate read role-play scripts (Table 5.11, Figure 5.2).

Table 5.11: Real cellular calls on one phone (12 November 2026)

Call	Expected	Result
Hindi digital arrest (fake police, Aadhaar parcel, arrest warrant, “tell no one”, RBI safe account)	STOP	STOP during the call; full-screen STOP card
English fake KYC (SBI KYC, account blocked, link by SMS, OTP)	STOP	STOP during the call; Auto chose English; type “Fake KYC / bank account block”
Genuine Hindi bank fraud check (card transaction, card blocked, “never share your OTP or PIN”)	no warning	No warning for the whole call

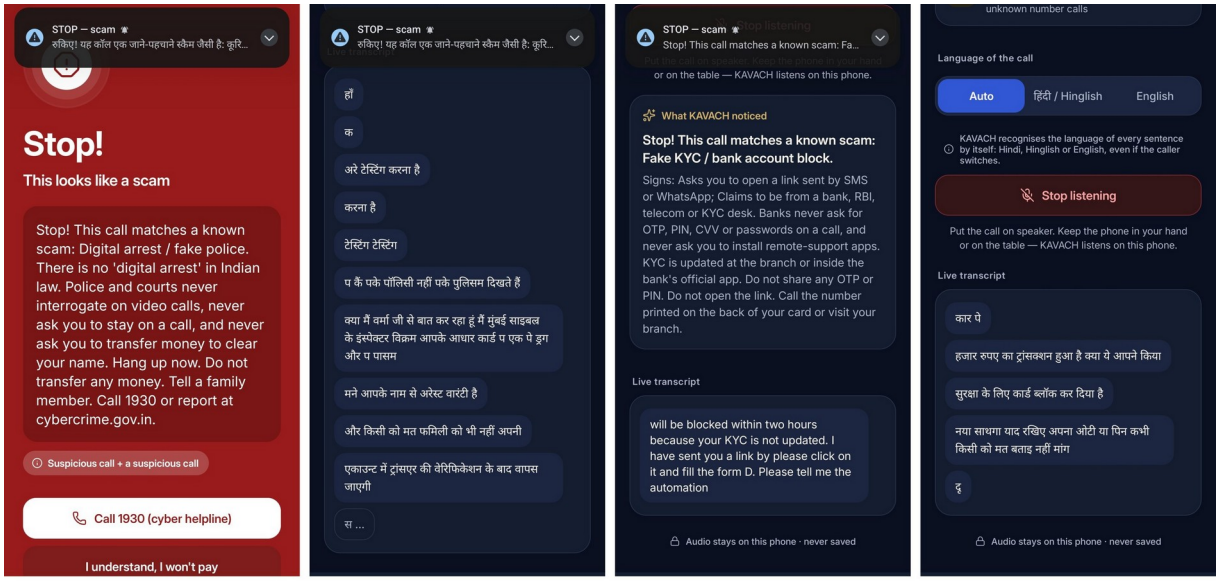


Figure 5.2: Screens during the real calls: Hindi digital arrest, English fake KYC and the genuine bank call

All three calls behaved as expected. Before this result, the first real call produced an empty transcript. The cause was that changing the default dialer made Android stop KAVACH and mark the call-listening service as crashed, while its switch still showed “on”; every recording during a call was then silenced. Turning the service off and on fixed it, and the app now detects both conditions and shows a Fix button.

5.11 The Llama Advisor

5.11.1 Export Verification

Table 5.12: Checks on the Llama export pipeline

Check	Result
Hugging Face → Meta weight conversion (tiny Llama 3.2-shaped model)	max |Δ logit| 3.6×10^{-7}
fp32 .pte with KV cache vs Hugging Face, step by step	max |Δ| 4.8×10^{-7}, 100 % next-token agreement
8da4w .pte vs Hugging Face	100 % next-token agreement
Real alert prompts	1,361-character constant prefix + 1.7–2.0 k-character suffix

5.11.2 Quality Before and After LoRA

The base and fine-tuned models were scored with models/llama/eval_llama.py on the first 100 development examples (Table 5.13, Figure 5.3).

Table 5.13: Llama 3.2 1B before and after KAVACH LoRA fine-tuning

Metric	Base Llama 3.2 1B	+ KAVACH LoRA
Valid JSON verdict	71 %	100 %
Scam / not scam correct	38 %	89 %
Scam type correct	58 %	76 %
Warning stage correct	71 %	100 %
Quoted evidence really in the transcript	8 %	96 %
Advice copied from the prompt’s example	32 %	0 %
Advice in the call’s language	55 %	100 %

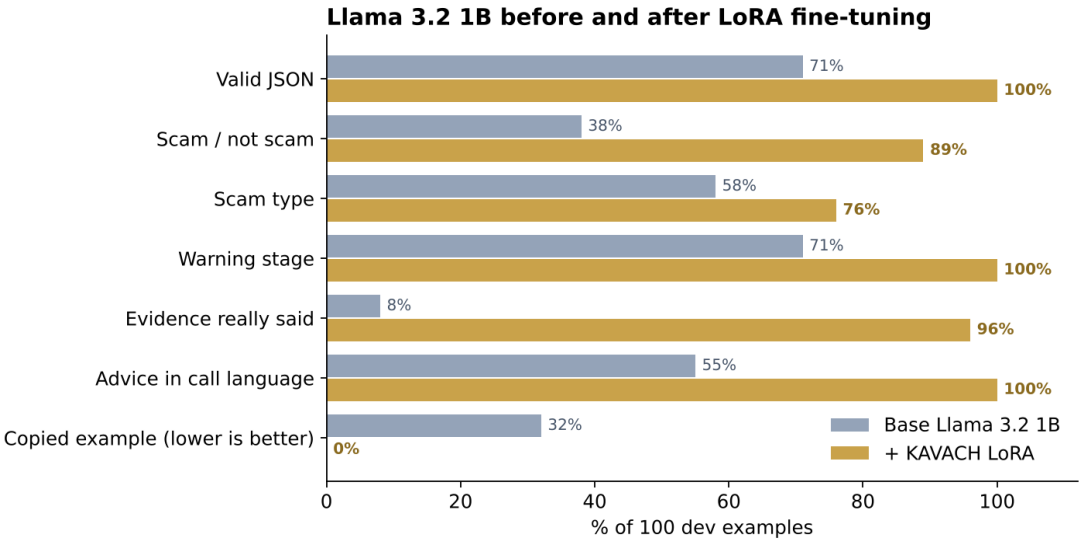


Figure 5.3: Llama quality metrics before and after LoRA fine-tuning

Fine-tuning fixed exactly the two problems seen on the phone before training: copied example advice (32 % to 0 %) and English advice in a Hindi call (55 % to 100 % in the right language). The development examples come from the same generator as the training data, and this score is for the full-precision merged model, so it is an upper bound; the locked held-out split was not used.

5.11.3 Llama on the Phone

The fine-tuned 4-bit .pte (820 MB, compared with 1,083 MB for Meta's SpinQuant file) loaded on the phone and ran the self-test call: verdict digital_arrest, confidence 0.91, with its own advice ("Hang up. Do not transfer any money. Tell a family member. Call 1930 or report at cybercrime.gov.in."), 92 tokens, first token 18.3 s cold and 11.5 tokens per second. During a replayed Hindi digital-arrest call it wrote its own Hindi advice with verbatim evidence ("साइबर क्राइम", "इंस्पेक्टर"), labelled on screen as explained by Llama 3.2 1B on this phone (Figure 5.4). The Hindi is understandable but not always polished, and in one earlier run on the same file the answer was replaced by the playbook text; the app now logs Llama's wording whenever that happens.

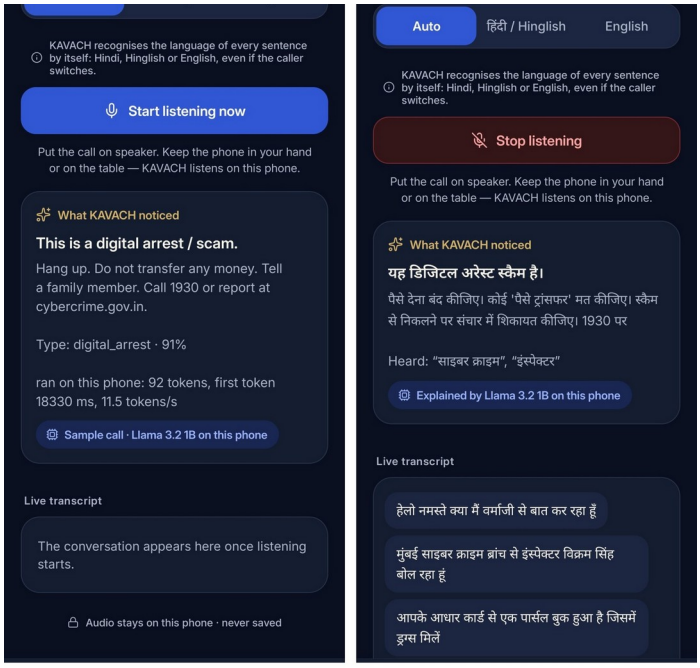


Figure 5.4: Llama 3.2 1B explaining a Hindi digital-arrest call on the phone

5.12 On-Phone Benchmark

All speed, battery and temperature figures were measured with the app's Benchmark screen on the team's phone, on battery: a Xiaomi 2312DRA50I with a Snapdragon 7s Gen 2 (four Cortex-A78 cores at 2.4 GHz and four Cortex-A55 cores at 1.96 GHz), 7.3 GB RAM and Android 15 (Table 5.14, Figure 5.5).

Table 5.14: On-phone benchmark (12 November 2026)

What	Result
Detector (rules + classifier + kill chain)	4.07 ms per sentence
Hindi IndicConformer (sentence)	RTF 0.153 at 2 threads, 0.092 at 4 (6.5× / 10.9× faster than real time)
English Parakeet 110M (sentence)	RTF 0.052 at 2 threads, 0.037 at 4 (19× / 27×)
Small streaming fallback models	Hindi RTF 0.239, English 0.117 at 2 threads
Auto listening, one minute only (VAD + both models, screen on)	54 % of one core, 385 mA whole-phone current (≈ 7.6 % battery per hour only if extrapolated), 38.0 → 37.9 °C, thermal status 0, 235 MB
Llama 3.2 1B, Meta SpinQuant INT4 file (ExecuTorch, XNNPACK)	Load 4.1 s, +1.1 GB; first token 12.9 s cold (768-token prompt), 5.9 s with warm prefix; prefill 56–60 tok/s; decode 11.6–11.9 tok/s; 858 mA while generating; no throttling
KAVACH LoRA 8da4w file (820 MB), self-test	First token 18.3 s cold, decode 11.5 tok/s

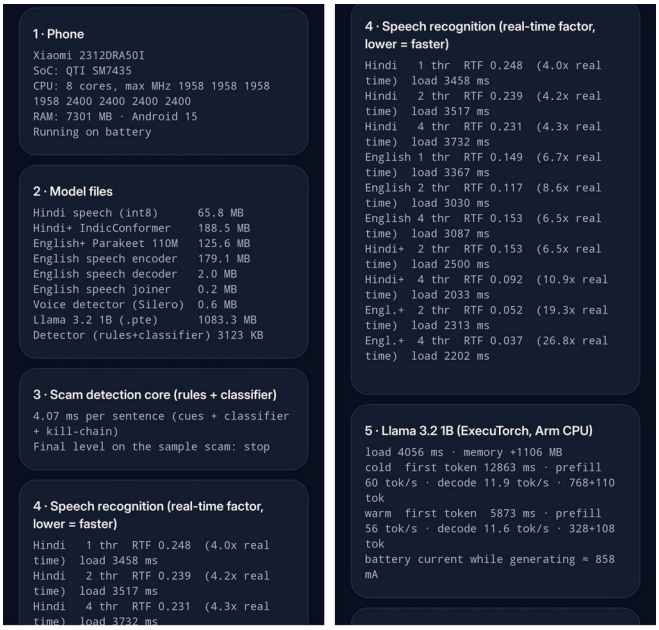


Figure 5.5: Benchmark screen on the phone

The Llama rows were measured with Meta's SpinQuant INT4 file before the fine-tuned file was ready; the KAVACH LoRA file decodes at 11.5 tokens per second in the on-phone self-test, so the two are on par. Both recognisers together use about a fifth of real time, so the transcript keeps up with the caller, and a one-minute listening run drew 385 mA for the whole phone with no rise in temperature. That would be under 8 % of the battery per hour if it held, but no sustained run or idle baseline has been measured yet, so this is an estimate, not a measured hourly drain. Llama is the slowest part, which is why its explanation always follows the instant rule-based warning instead of delaying it. Four threads make the big speech models clearly faster; the app uses two to save battery, but whether two threads really use less energy has not been measured. For reference, ExecuTorch reports about 50 tokens per second for Llama 3.2 1B SpinQuant on a flagship phone, so the mid-range result of about 12 tokens per second leaves room for the KleidiAI and NPU work planned for Phase 2.

5.13 Network Watch

Table 5.15: Network Watch results

Part	Test	Result
App traffic watch	200 simulated 2-hour traces per scenario, 30 s polling	Spyware uploads, slow leaks and heartbeats: 100 % detected (uploads in 30–90 s, heartbeats in 10–35 min); chat, video, streaming, backups: 0 % false alarms
Domain model	6 DGA families never seen in training	cryptolocker 92 %, necurs 92 %; dictionary-word families ≈ 0 % (blind spot); 0.36 % false alarms on 85k popular domains; 0 genuine Indian domains flagged

Part	Test	Result
Flow model, UNSW-NB15	82,332 test flows	Precision 0.939, recall 0.885, F1 0.911, 7.1 % false alarms
Flow model, NSL-KDD	KDDTest+ used once	F1 0.854; new attack types 65 % vs seen 82 %
Flow model on phone-like traffic	Replayed capture	Flags 60/60 normal HTTPS sessions: UNSW-NB15 has no normal TLS flows, so the model raises no alerts
Pcap replay demo	Browsing + scan + DGA + look-alike + heartbeat + 2.4 MB upload	Rules find all 5; no alert on normal browsing
Home Wi-Fi check	10 abused services on the /24	254 addresses in 12 s; found an open ADB port and explained the fix

The flow model is a useful negative result: a model with a good F1 on a public benchmark flagged every normal HTTPS session from a phone, because its training data contains no normal encrypted traffic. It is therefore kept for research only, and alerts come from the rule-based traffic and DNS checks.

5.14 Problems Found and Fixed

Table 5.16: Problems found during testing and how they were fixed

Problem	Cause	Fix
Scam calls stopped at yellow on the phone	Spoken yellow warning muted the microphone for 19–33 s	Yellow warnings vibrate only
Hindi scams missed in noise	Small streaming Hindi model	IndicConformer, sentence by sentence
Weak English on Indian terms (“arbi” for RBI)	Small generic English model	Parakeet 110M; spelled letters joined (“o t p”)
Language had to be chosen by hand	One model per call	Auto mode with LangPick
Empty transcript on a real call	Accessibility service marked crashed after dialer change	Health checks and a Fix button
Llama copied the prompt’s example	Base model, no fine-tuning	LoRA; copied advice replaced and labelled
Half-finished sentences in advice	Generation cut at the token limit	Validator keeps whole sentences only
“scam”: “maybe” read as not a scam	Loose JSON parsing	Strict validation and one re-prompt
Python and Kotlin brand order differed	Hash randomisation in Python	Deterministic ordering; parity test
Colab training failed	torchao version conflict with PEFT	torchao removed in the notebook

5.15 Targets and Achievements

Table 5.17: Target specifications compared with what was achieved

Parameter	Target	Achieved	Met?
Network use	None	No INTERNET permission	Yes
Audio storage	None	Memory only	Yes
Languages	Hindi, Hinglish, English, automatic	Auto, 92.7 % per sentence	Yes
Detector latency	< 10 ms	4.07 ms	Yes
Speech speed	Faster than real time	≈ 5× faster, both models	Yes
Median time to STOP	< 30 s	12.6–21.3 s	Yes
False STOP	≤ 2 %	0 observed (0/489 synthetic, 0/24 human-written)	Not shown
STOP before money	≥ 90 %	50–82 %	No
Explanation on the Arm CPU	Llama 3.2 1B, 4-bit	11.5–11.9 tok/s	Yes
Battery	< 10 % per hour	≈ 7.6 % per hour, one-minute estimate	Partly
Real calls	Correct on one phone	3 / 3	Yes

5.16 Summary

Of the eleven Phase 1 targets, eight are met, the battery target is met only as a one-minute estimate, the 2 % false-STOP target cannot be shown with so few human-written genuine calls even though none was stopped, and catching nine in ten scams before the payment step is missed. Three real cellular calls behaved as expected. Fusion adds STOPs at the risky action but no earlier warnings. The 20-call human pilot, a sustained battery run and scoring the deployed 4-bit Llama are the next measurements.

CHAPTER 6
LIMITATIONS, ETHICS AND THREAT MODEL

6.1 Introduction

A system that listens to calls and warns people must be honest about where it can fail and careful about how it could be misused. This chapter lists the known limitations, the ways a scammer might try to avoid KAVACH, and the ethical choices made in the design.

6.2 Limitations

- Speech recognition is the largest source of error. Hindi WER on real speaker-phone calls with human voices is not yet measured; most audio results use synthetic TTS voices, which share one voice between training and testing.
- A television or other people talking near the phone is the hardest case; the recogniser may transcribe the background instead of the caller.
- The call must be on speaker unless a phone maker enables system mode, because the microphone hears the room.
- Training data is synthetic and written by the team. The hand-written and held-out sets reduce but do not remove author bias; the human-recorded test set planned for 21 November is the real measure.
- Catching scams before the payment step is at 50–82 %, below the 90 % target; false CAUTIONs on human-written genuine calls are frequent, and Llama cannot reduce them because it only escalates.
- Fusion links events by time only, so an unrelated scam message during a genuine call can be blamed on the call (32 of 40 in the ablation).
- The deployed 4-bit Llama has only been observed on the phone; its quality was scored in full precision on generator data, and its confidence is self-reported, not calibrated. Resistance to spoken prompt injection rests on the rules of the contract, not on a measured attack rate.
- Battery use is a one-minute whole-phone sample with no idle baseline; the 30-minute soak is not run yet.
- Llama’s Hindi is understandable but not always polished, and its explanation arrives several seconds after the rule-based warning.
- Network Watch rule numbers come from simulated traces, and the flow model needs real phone traffic before it can raise alerts.
- App Inspector and Screen-share Guard are previews and have not been evaluated on public malware data.

6.3 Threat Model

Table 6.1: Ways a scammer may try to avoid KAVACH and the defences

Evasion	Defence
Avoid trigger words	Classifier and cue-category combinations generalise beyond exact phrases; Llama reads meaning
Keep the talk clean, push a link or APK	Cross-channel rules: an unknown call plus a dangerous link, APK or screen share is STOP
Calm the victim with small talk	The level is sticky within a session
Switch language mid-call	Auto mode chooses the language per sentence
Ask the victim to switch off speaker	KAVACH Phone keeps unknown calls on speaker; a STOP card stays on screen
Call from a saved or spoofed contact	Shield still checks links, apps and payments during the call
Prompt injection through speech	Llama can never lower or clear a warning; its quotes must be verbatim; the validator enforces the contract (not yet tested with real injection attempts)

6.4 Ethics and Privacy

- Consent: KAVACH listens only to calls the user has chosen to protect, only on speaker, and only from numbers not in the contacts; every permission is explained before Android asks for it.
- Data minimisation: no audio is stored, the transcript stays in memory, and the main app cannot use the network.
- Test recordings: role-play calls are recorded only with the written consent of every speaker, and no real victim audio is used.
- False alarms: a false STOP on a genuine bank call could stop someone from acting on a real fraud alert, which is why zero false STOPs is a hard target and genuine safety advice is a separate cue category.
- Accessibility use: the call-listening service reads no screen content and performs no actions; a store version would need the phone maker’s system mode or a declared accessibility use case.
- Human control: KAVACH warns and suggests; it never ends a call, blocks a payment or contacts anyone without the user’s choice.

6.5 Summary

KAVACH’s main risks are recognition errors on real voices, the speaker requirement and synthetic training data. The design keeps the user in control and keeps all data on the phone, and every known weakness is stated so that Phase 2 can measure it.

CHAPTER 7
CONCLUSION AND FUTURE WORK

7.1 Conclusion

This report presented KAVACH, an Android application that detects a scam phone call while it is happening and explains it in the caller's language, entirely on the phone. It listens to a call on speaker through Android's accessibility audio exception, recognises each sentence with IndicConformer for Hindi and Parakeet for English, chooses the language automatically, scores the conversation with a 4 ms detector, joins it with evidence from messages, links, apps and the network, and asks a LoRA-fine-tuned, 4-bit Llama 3.2 1B on the Arm CPU to explain the alert.

On a mid-range Snapdragon 7s Gen 2 phone, speech recognition in both languages runs about five times faster than real time, a one-minute listening run drew 385 mA for the whole phone and Llama decodes about 12 tokens per second. Detection reached STOP on 93.4 % of scams in unseen synthetic calls with no genuine call stopped in any set (a sample too small to prove a rate), Hindi telephone-band detection rose from 0 % to 58 % with IndicConformer, LoRA raised Llama's scam accuracy from 38 % to 89 %, and three real cellular calls behaved exactly as expected. Joining channels adds STOPs at the risky action but no earlier warnings, and can wrongly blame a genuine call. The main open problems are catching more scams before the payment step, cutting false CAUTIONs, and linking events by the organisation involved rather than by time alone.

7.2 Future Work (Phase 2)

Table 7.1: Phase 2 plan

Work item	Goal
Human-recorded test set	100+ scam and 100+ genuine consented role-play calls; Hindi WER on speaker-phone audio; adapt the recognisers
Llama escalation effect	Measure how much Llama raises STOP before payment and lowers false CAUTIONs on every call set
Arm optimisation	Quantisation ladder with KleidiAI on and off, big vs LITTLE cores, a budget phone, a 30-minute thermal soak
NPU	Export the speech models to the phone's NPU through ExecuTorch and compare energy with the CPU
Better Hindi advice	More Hindi training pairs and a review of Llama's Hindi wording
App and network models	App Inspector on public malware data; flow model retrained on Android traffic; dictionary-DGA detection
Pilot	A consent-based pilot with 20–30 family volunteers

KAVACH shows that the Arm processor already inside an ordinary Indian phone is enough to hear a scam, recognise it and explain it in Hindi or English, without sending a single word off the device.

REFERENCES

[1] Scroll, “Indians lost Rs 22,845 crore to cyber fraud in 2024, a 206% rise from previous year: Centre,” Jul. 2025. [Online]. Available: https://scroll.in/latest/1084796/indians-lost-rs-22845-crore-to-cyber-fraud-in-2024-a-206-rise-from-previous-year-centre

[2] Inc42, “Indians lost INR 1,935 Cr to digital arrest scams in 2024: Govt,” Mar. 2025. [Online]. Available: https://inc42.com/buzz/indians-lost-inr-1935-cr-to-digital-arrest-scams-in-2024-govt

[3] E. M. Hutchins, M. J. Cloppert, and R. M. Amin, “Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains,” Lockheed Martin Corp., Tech. Rep., 2011.

[4] A. Gulati et al., “Conformer: Convolution-augmented transformer for speech recognition,” in Proc. Interspeech, 2020, pp. 5036–5040.

[5] D. Rekesh et al., “Fast Conformer with linearly scalable attention for efficient speech recognition,” in Proc. IEEE ASRU, 2023.

[6] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks,” in Proc. ICML, 2006, pp. 369–376.

[7] T. Javed et al., “IndicVoices: Towards building an inclusive multilingual speech dataset for Indian languages,” in Findings of ACL, 2024.

[8] NVIDIA, “parakeet-tdt_ctc-110m,” model card, 2025. [Online]. Available: https://huggingface.co/nvidia/parakeet-tdt_ctc-110m

[9] k2-fsa, “sherpa-onnx: speech recognition, synthesis and VAD with ONNX Runtime,” 2025. [Online]. Available: https://github.com/k2-fsa/sherpa-onnx

[10] Silero Team, “Silero VAD: pre-trained enterprise-grade voice activity detector,” 2024. [Online]. Available: https://github.com/snakers4/silero-vad

[11] A. Grattafiori et al., “The Llama 3 herd of models,” arXiv:2407.21783, 2024.

[12] Z. Liu et al., “SpinQuant: LLM quantization with learned rotations,” arXiv:2405.16406, 2024.

[13] PyTorch Team, “ExecuTorch: On-device AI inference for PyTorch,” 2025. [Online]. Available: https://github.com/pytorch/executorch

[14] Arm Ltd., “KleidiAI: micro-kernels for AI workloads on Arm CPUs,” 2025. [Online]. Available: https://gitlab.arm.com/kleidi/kleidiai

[15] E. J. Hu et al., “LoRA: Low-rank adaptation of large language models,” in Proc. ICLR, 2022.

[16] K. Weinberger, A. Dasgupta, J. Langford, A. Smola, and J. Attenberg, “Feature hashing for large scale multitask learning,” in Proc. ICML, 2009, pp. 1113–1120.

[17] Android Developers, “Share audio input,” 2025. [Online]. Available: https://developer.android.com/media/platform/sharing-audio-input

[18] R. Speer, “wordfreq: Access a database of word frequencies in many languages,” 2022. [Online]. Available: https://github.com/rspeer/wordfreq

[19] Reserve Bank of India, circular RBI/2025-26/28 on the exclusive “.bank.in” domain for banks, 2025.

[20] N. Moustafa and J. Slay, “UNSW-NB15: a comprehensive data set for network intrusion detection systems,” in Proc. Military Communications and Information Systems Conf. (MilCIS), 2015.

[21] T. A. Almeida, J. M. G. Hidalgo, and A. Yamakami, “Contributions to the study of SMS spam filtering: New collection and results,” in Proc. ACM DocEng, 2011.

[22] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, “A detailed analysis of the KDD CUP 99 data set,” in Proc. IEEE CISDA, 2009.

[23] Rhasspy, “Piper: a fast, local neural text to speech system,” 2024. [Online]. Available: https://github.com/rhasspy/piper

ANNEXURE A: INSTALLING AND SETTING UP KAVACH

1. Build the debug APKs with scripts/build_apks.sh (they appear in dist/), or download them from the GitHub Actions run.
2. Connect the phone with USB debugging on and install: adb install dist/kavach-guardian-debug.apk (and optionally dist/kavach-netwatch-debug.apk).
3. Copy the speech models with scripts/push_models.sh --asr (VAD, IndicConformer, Parakeet and the fallback models), then Llama with scripts/push_models.sh --llama <file.pte> <tokenizer.model>.
4. Open KAVACH and grant the microphone, phone-state, notification-access and usage-access permissions when asked.
5. Calls → Protect my calls: turn on the KAVACH call-listening accessibility service.
6. Choose the call language (Auto is the default) and check that every row on the Readiness screen is green.
7. Setup & tools → Test on-device AI runs the Llama self-test; Benchmark measures the phone.

ANNEXURE B: REPRODUCING THE RESULTS

```
pip install numpy scipy scikit-learn pytest
./data/public/download.sh
python data/synth/generate_calls.py && python data/synth/generate_sms.py
python data/dev/build_handwritten.py
python models/classifiers/train_text.py && python
models/classifiers/train_url.py
python bench/eval_calls.py --split test        # also dev / handwritten /
heldout
python -m pytest -q tests && python tests/make_parity_vectors.py
cd android && ./gradlew :core:test             # Kotlin parity (336 checks)
python bench/netwatch_sim.py && python bench/netwatch_pcap_demo.py
python bench/e2e_noise.py --asr zipformer-en-2023-06-21 --configs base hotwords
# Llama: open notebooks/kavach_llama_lora_colab.ipynb in Colab (T4) and run all
```

ANNEXURE C: LLAMA OUTPUT CONTRACT

Example of a valid verdict returned by Llama for a digital-arrest call:

```
{"scam": true, "type": "digital_arrest", "stage": "isolation",
 "confidence": 0.91,
 "evidence": ["cyber crime", "do not tell anyone"],
 "say": "Hang up. Do not transfer any money. Tell a family member. Call 1930.",
 "action": "hang_up"}
```

Allowed stages: none, contact, hook, isolation, action, payment. Allowed actions: none, continue_with_care, verify_independently, do_not_share, do_not_pay, hang_up. Allowed types: the twelve identifiers of Table 1.1 or none.

ANNEXURE D: BENCHMARK OUTPUT FIELDS

The Benchmark screen writes one JSON file (bench/results/device/kavach_bench_20261112_013528.json for the results in Chapter 5). Its main fields are:

Table D.1: Main fields of the benchmark JSON file

Field	Meaning
phone	Model, SoC, ABI, cores and their maximum clocks, RAM, Android version, charging state
model_files_mb	Size of every model file on the phone
detector_ms_per_segment	Detector time per sentence
asr.<model>_t<n>	Real-time factor, load time and memory of each speech model at 1, 2 and 4 threads
llama.cold / llama.warm	Prompt and generated tokens, time to first token, prefill and decode speed, total time
llama.load_ms, pss_*_mb	Llama load time and memory before, after loading and while running
llama.battery_current_ma_*	Battery current when idle and while generating
listening_60s	One minute of Auto listening: sentences, CPU share, current, battery per hour, temperatures
measured_at	Date and time of the run

ANNEXURE E: TEAM CONTRIBUTIONS

All three students worked across the project. The main areas of each are listed below.

Table E.1: Main areas of work

Name	Main areas of work
Siddhant Singh	Call listening: on-device speech recognition, Auto language, Call Guardian and the Android app
Bogavalli Sai Akhilash	Detection and Llama: scam playbook, training data, classifiers, LoRA fine-tuning and evaluation
Shivansh Bose	Shield and Network Watch: messages, links, apps, payments, network and Wi-Fi checks, privacy and security review
Dr. Rahul Ratnakumar	Faculty Mentor